BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Thông tin sinh viên	Nội dung
Họ và tên	Đặng Ngọc Như Ý
MSSV	3123410438
Lớp	DTN1241

PHẦN I. BÀI TẬP TÌM KIẾM TRONG AI

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.

• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.

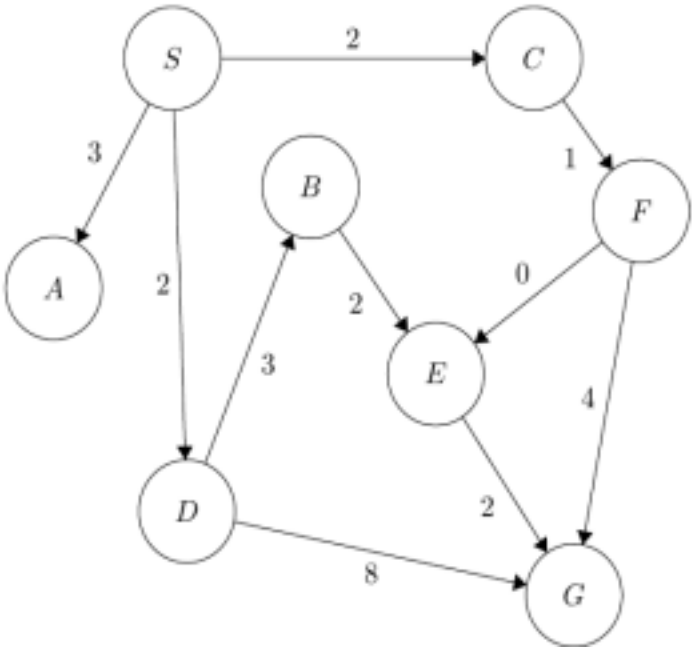
• Thiết kế và phân tích heuristic cho A*.

• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.

• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	{S, A, C, D, F, B}
Path	S -> D -> G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	{S, A, C, F, E}
Path	S -> C -> F -> E -> G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G

Path cuối cùng: S -> D -> G

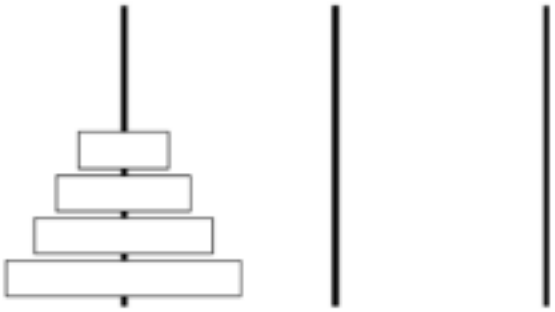
Total Cost: 2 + 8 = 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS sử dụng hàng đợi (FIFO) để khám phá các nút nông trước nên đảm bảo tìm được đường đi ngắn nhất về số cạnh (tối ưu về bước đi), trong khi DFS sử dụng ngăn xếp (LIFO) để đi sâu vào từng nhánh trước nên không đảm bảo tính tối ưu về đường đi hoặc chi phí.

Bài 2. Tháp Hà Nội – Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Dùng một bộ ba danh sách tương ứng với ba cọc (P_1, P_2, P_3) để lưu thứ tự các đĩa trên mỗi cọc, giúp xác định chính xác vị trí từng đĩa và các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Với n đĩa và 3 cọc, tổng số trạng thái là 3^n. Riêng với trường hợp 4 đĩa như hình vẽ, không gian trạng thái gồm $3^4 = 81$ trạng thái

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

Dựa vào hình vẽ ban đầu, toàn bộ 4 đĩa đều nằm chồng lên nhau ở cọc thứ nhất theo thứ tự từ lớn ở dưới đến nhỏ ở trên, trong khi hai cọc còn lại đang trống. Do đó, trạng thái khởi đầu được viết gọn là $([4, 3, 2, 1], [], [])$

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Nhấc đĩa ở đỉnh một cọc bất kỳ đặt lên đỉnh cọc khác. Điều kiện: cọc nguồn không rỗng, và đĩa chuyển sang phải nhỏ hơn đĩa ở đỉnh cọc đích (hoặc cọc đích trống).

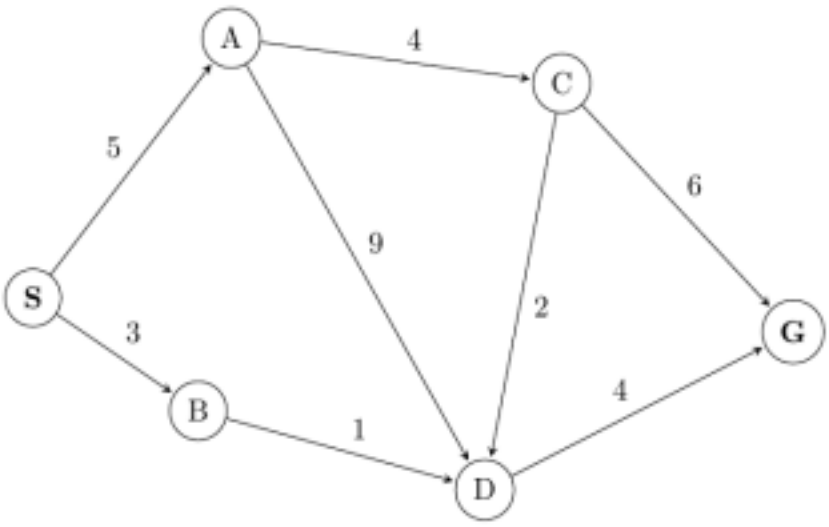
2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Hàm kiểm tra mục tiêu sẽ trả về giá trị đúng khi toàn bộ các đĩa đã được chuyển gọn gàng sang cọc thứ ba theo đúng thứ tự kích thước, đồng thời cọc thứ nhất và cọc thứ hai hoàn toàn trống rỗng, tức là đạt trạng thái $([], [], [4, 3, 2, 1])$

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Khoảng cách ngắn nhất từ G đến các nút

Node	h(n)
S	8
A	9
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: $\{S, B, D, G\}$

Path: $S \rightarrow B \rightarrow D \rightarrow G$

Total Cost: $3 + 1 + 4 = 8$

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) ≤ c(n,n') + h(n').

- h(S) = 9, h(A) = 9, h(B) = 5, h(C) = 6, h(D) = 4, h(G) = 0
- Cạnh phạm vị: xét cạnh S -> B có chi phí c(S, B) = 3

Ta có điều kiện consistent: h(S) ≤ c(S, B) + h(B) ⇔ 9 ≤ 3 + 5 = 8 (Sai)

=> Cạnh S -> B vi phạm tính consistent dù vẫn thỏa mãn tính

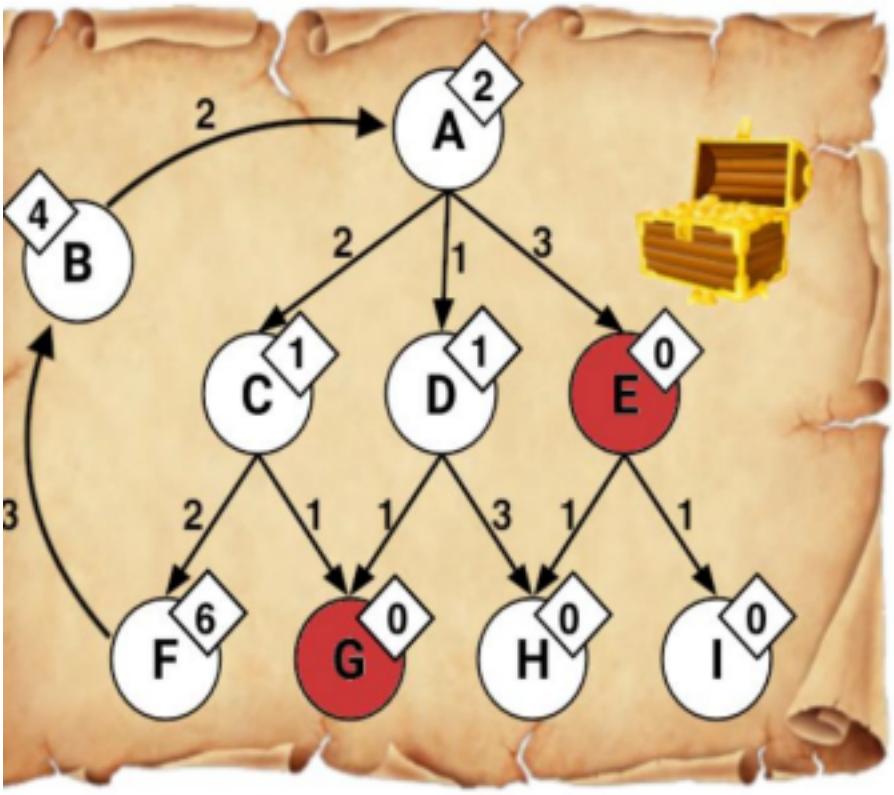
3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Heuristic consistent luôn admissible vì nó đảm bảo ước lượng khoảng cách từ một nút đến đích không bao giờ vượt quá chi phí thực tế tối ưu. Ngược lại, heuristic admissible chưa chắc consistent vì nó chỉ ràng buộc tổng chi phí chứ không ép buộc mối quan hệ khoảng cách giữa các nút liền kề trên đồ thị

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set: $\{A, C, F, B, G\}$ **DFS:** $A \rightarrow C \rightarrow F \rightarrow B \rightarrow A \rightarrow G$

Path returned: $A \rightarrow C \rightarrow F \rightarrow B \rightarrow G$

4.2. Breadth-First Search

Explored Set: $\{A, C, D, E\}$

Path returned: $A \rightarrow E$

4.3. Uniform-Cost Search

Explored Set: $\{A, D, C, G\}$

Path returned: $A \rightarrow D \rightarrow G$

Cost: 1 + 1 = 2

4.4. Greedy Search

Explored Set: $\{A, E\}$

Path returned: $A \rightarrow E$

Cost: 3

4.5. A* Search

Explored Set: $\{A, D, G\}$

Path returned: $A \rightarrow D \rightarrow G$

Cost: 1 + 1 = 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	$A \rightarrow C \rightarrow F \rightarrow B \rightarrow G$	
BFS	Độ sâu nhỏ nhất	$A \rightarrow E$	3
UCS	g(n)	$A \rightarrow D \rightarrow G$	2
Greedy	h(n)	$A \rightarrow E$	3
A*	g(n) + h(n)	$A \rightarrow D \rightarrow G$	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai:

Giải thích:

DFS không có cơ chế tối ưu hoặc định hướng bằng chi phí/heuristic nên nó có thể đi lạc vào một nhánh rất sâu và tìm thấy đích hoặc cụt đường mà chỉ mở rộng rất ít nút (hoặc ngược lại, có thể mở rộng nhiều hơn). Không có quy luật bắt buộc DFS phải mở rộng nhiều nút hơn A*

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sa

Giải thích:

Một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột chỉ trong một nước đi duy nhất (nếu đường đi không bị chặn). Do đó, số nước đi tối thiểu từ ô A đến ô B thực tế chỉ là 1 nước (nếu cùng hàng/cột) hoặc tối đa là 2 nước (nếu khác hàng và khác cột). Trong khi đó, khoảng cách Manhattan có thể lớn hơn 2 (ví dụ từ $(0, 0)$ đến $(3, 4)$ thì khoảng cách Manhattan là $3 + 4 = 7$). Vì giá trị ước lượng của khoảng cách Manhattan lớn hơn chi phí thực tế tối đa (vượt quá 1 hoặc 2 nước đi), nó đã đánh giá cao hơn thực tế và không phải là admissible

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích:

Khoảng cách Euclidean là đường chim bay ngắn nhất giữa hai điểm. Trong bài toán tìm đường, chi phí thực tế đi qua các ô lưới bao giờ cũng lớn hơn hoặc bằng đường chim bay trực tiếp, do đó nó không bao giờ đánh giá cao hơn chi phí thực tế (thỏa mãn tính admissible)

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.
Đúng / Sai: Sai

Giải thích:

Tổng của các heuristic admissible có thể vượt quá chi phí thực tế ngắn nhất (ví dụ h_1 và h_2 sát với chi phí thực tế thì $h_1 + h_2$ sẽ lớn hơn chi phí thực tế), do đó làm mất tính admissible. (Tuy nhiên, hàm $\max(h_1, h_2)$ mới luôn là admissible)

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.
Đúng / Sai: Sai

Giải thích:

Tính admissible chỉ đảm bảo ước lượng từ một nút đến đích không vượt quá chi phí thực tế, nhưng không ràng buộc mối quan hệ khoảng cách giữa các nút liền kề nhau trên đồ thị. Do đó, một heuristic admissible vẫn có thể vi phạm bất đẳng thức tam giác và không consistent

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.
Đúng / Sai: Sai

Giải thích:

Để A* sử dụng thuật toán tìm kiếm trên đồ thị (Graph Search) đảm bảo tính tối ưu, heuristic bắt buộc phải consistent (nhất quán). Nếu heuristic chỉ admissible (không nhất quán), A* với Graph Search có thể bỏ qua đường đi tối ưu nếu một nút được mở rộng lần đầu qua đường đi đắt hơn nhưng nằm trong explored set.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n) + h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

PHẦN II. BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Mục tiêu học tập

• Mô hình hóa bài toán tìm kiếm trong AI.

• Vận dụng BFS, UCS, Greedy và A*.

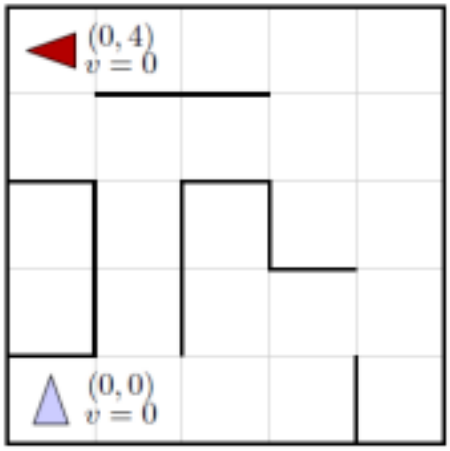
• Phân tích và đánh giá heuristic.

• Phân biệt heuristic admissible và consistent.

• So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Bài 1. Tìm kiếm và heuristic

Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.



Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v − 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm v < 0 hoặc v > Vmax.

• Giữ tốc độ khi v = 0.

• Rẽ khi v ≠ 0.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Tác tử bắt đầu tại trạng thái $(0, 0, Bắc, 0)$ và cần đến trạng thái đích $(0, 4, Tây, 0)$, đi theo nhánh ngoài cùng bên trái và qua ô $(1, 2)$.

Chuỗi hành động phải tuân thủ các quy tắc: chỉ được rẽ khi vận tốc bằng 0; vận tốc sau khi điều chỉnh phải nằm trong khoảng từ 0 đến $Vmax$; không được di chuyển xuyên qua tường; và trạng thái cuối phải có đúng vị trí, hướng đích và vận tốc bằng 0.

Chuỗi hành động tối ưu: Tăng tốc, giữ tốc độ, giảm tốc, rẽ trái, tăng tốc, giữ tốc độ, giảm tốc, rẽ trái (tùy thuộc vào tọa độ chi tiết của tường trên sơ đồ mê cung)

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Với lưới kích thước $M \times N$, ta có:

- Vị trí (x, y): có MN khả năng.
- Hướng di chuyển d: có 4 khả năng (Bắc, Nam, Đông, Tây).
- Vận tốc v: có $Vmax + 1$ giá trị nguyên từ 0 đến $Vmax$.

Tổng kích thước không gian trạng thái là: $4MN(Vmax + 1)$

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

Có. Một trạng thái hợp lệ có thể không có nút con nếu tất cả hành động có thể thực hiện từ trạng thái đó đều bị vi phạm (ví dụ: xe đứng trước bức tường, các hướng xung quanh đều bị chặn và không thỏa mãn điều kiện rẽ hoặc di chuyển tiếp)

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất là 5, tương ứng với tập 5 hành động có thể sinh ra từ tác tử: *Faster, Slower, Maintain, Turn Left, Turn Right*. Mặc dù tại mỗi trạng thái cụ thể một số hành động sẽ bị loại bỏ do ràng buộc vận tốc ($v = 0$ hay $v > 0$), nhưng hệ số phân nhánh tối đa của toàn bộ mô hình bài toán vẫn là 5

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

Không hoàn toàn/Tùy trường hợp. Khoảng cách Manhattan tính số ô cần đi theo phương ngang và dọc, nhưng trong bài toán động học có thêm ràng buộc về vận tốc và quán tính, khoảng cách Manhattan đơn thuần có thể không phản ánh chính xác số bước hành động thực tế tối thiểu.

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

Có. Heuristic này là admissible, với điều kiện Vmax > 0 và mỗi hành động di chuyển tối đa Vmax ô Mỗi hành động di chuyển có thể làm giảm khoảng cách Manhattan tối đa $Vmax$ ô. Do đó, số hành động tối thiểu cần thiết để đến đích ít nhất là $D_{Manhattan}/Vmax$. Các hành động rẽ và điều chỉnh vận tốc có thể làm tăng số hành động thực tế, nhưng không khiến chi phí tối ưu nhỏ hơn cận dưới này. Vì vậy, heuristic thỏa mãn điều kiện $h(s) \leq h^{*}(s)$, nên đây là một heuristic chấp nhận được (admissible)

g) Nếu dùng heuristic không admissible trong A Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.*

Không. Với đồ thị hữu hạn, hệ số phân nhánh hữu hạn và heuristic không âm, A* Tree Search vẫn đầy đủ nếu chi phí mỗi bước dương (hoặc được giới hạn dưới bởi một hằng số dương). Heuristic không admissible có thể làm mất bảo đảm tìm được lời giải tối ưu, nhưng không làm mất tính đầy đủ trong các điều kiện trên

h) Nếu dùng heuristic không admissible trong A Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.*

Có thể thay đổi. Khi heuristic không admissible (có thể đánh giá cao hơn chi phí thực tế ở một số nhánh), A* không còn đảm bảo tìm được đường đi có chi phí tối ưu nữa

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☑ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☑ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Bài 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS)

Bất đẳng thức: Tập các nút được mở rộng trước khi kết thúc bao gồm toàn bộ các nút nông hơn hoặc có độ sâu bằng độ sâu của nút đích nông nhất n_s (tức là mọi nút n thỏa mãn độ sâu $d(n) \leq d(n_s)$)

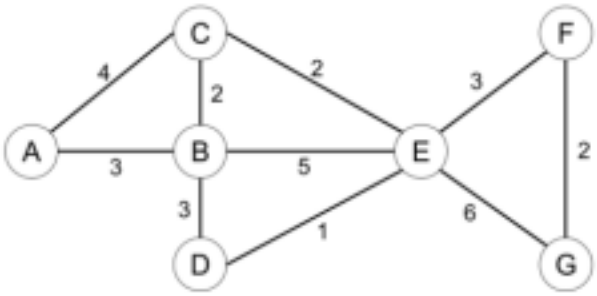
b) Uniform-Cost Search (UCS)

Bất đẳng thức: Tập các nút được mở rộng trước khi kết thúc bao gồm mọi nút n có chi phí tích lũy nhỏ hơn hoặc bằng chi phí của nút đích tối ưu n_c (tức là $g(n) \leq g(n_c)$)

c) A* Search, giả sử h là heuristic nhất quán

Bất đẳng thức: Tập các nút được mở rộng trước khi kết thúc bao gồm mọi nút n có giá trị hàm đánh giá tổng quát nhỏ hơn hoặc bằng chi phí tối ưu thực tế (tức là $f(n) = g(n) + h(n) \leq g(n_c)$)

Bài 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Bảng heuristic:

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	☑	☐	
Greedy với h1	☐	☐	☐	A-B-D-E-G
Greedy với h2	☑	☐	☑	
A* với h1	☐	☐	☐	
A* với h2	☐	☑	☐	

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án:

Đường đi: $A \to C \to E \to F \to G$

=> Chi phí: 4 + 2 + 3+ 2 = 11

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☑ Không Consistent: ☐ Có ☑ Không

Vì $h_1(A) = 12$, trong khi chi phí thực tế ngắn nhất từ A đến G chỉ là 11, do đó nó đánh giá cao hơn thực tế và không admissible

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☑ Có ☐ Không Consistent: ☐ Có ☑ Không

Giá trị ước lượng của tất cả các nút đều nhỏ hơn hoặc bằng chi phí thực tế ngắn nhất từ nút đó đến đích

Vi phạm điều kiện nhất quán ở cạnh $A - C$, cụ thể $h_2(A) = 11$, trong khi

$c(A, C) + h_2(C) = 4 + 6 = 10$, dẫn đến 11 >

Bài 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

1. Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ☑ Sai

Nếu đồ thị có chu trình hoặc nhánh vô hạn, DFS Tree Search có thể bị kẹt trong vòng lặp vô hạn và không tìm được đích, do đó không đảm bảo tính đầy đủ (complete)

2. Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

BFS duyệt theo từng mức độ sâu tăng dần và không gian trạng thái hữu hạn, nên luôn đảm bảo tìm thấy đích nếu tồn tại đường đi

3. Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☑ Đúng ☐ Sai

Kết hợp ưu điểm của DFS và BFS, Iterative Deepening duyệt vét cạn theo từng giới hạn độ sâu nên hoàn toàn đầy đủ trên đồ thị hữu hạn

4. Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ☑ Sai

Nhờ có tập explored set loại bỏ các trạng thái trùng lặp, frontier của Graph Search thường nhỏ hơn hoặc bằng Tree Search

5. Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.

☑ Đúng ☐ Sai

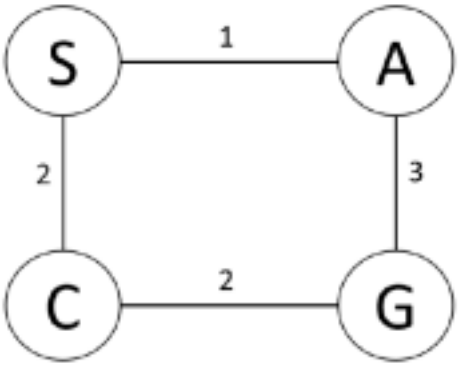
Đây là tính chất kinh điển của thuật toán IDS (thời gian tương đương BFS do duyệt lại các mức, còn bộ nhớ theo chiều sâu của DFS)

6. Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.

☑ Đúng ☐ Sai

Hàm lấy giá trị nhỏ nhất của các heuristic consistent vẫn giữ nguyên tính chất thỏa mãn bất đẳng thức tam giác và tính consistent

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

Trạng thái	h(s)
S	4
A	3
C	1
G	0

Chi phí thực tế ngắn nhất từ các nút đến G:

- Từ S: $\min(1 + 3, 2 + 2) = 4$ (đường $S \to C \to G$ hoặc $S \to A \to G$).
- Từ A: 3 (đường $A \to G$).
- Từ C: 2 (đường $C \to G$).

Đề xuất bộ giá trị $h(s)$ admissible nhưng không consistent:

- Để admissible, giá trị heuristic của mỗi nút không được vượt quá chi phí thực tế ngắn nhất đến G:
  - $h(S) = 3$ (nhỏ hơn 4 → admissible)
  - $h(A) = 3$ (bằng 3 → admissible)
  - $h(C) = 1$ (nhỏ hơn 2 → admissible)
  - $h(G) = 0$
- **Kiểm tra tính consistent:** Xét cạnh $S \to C$ có chi phí $c(S, C) = 2$. Điều kiện consistent yêu cầu $h(S) \leq c(S, C) + h(C)$.
  - Thay số: $3 \leq 2 + 1 \Leftrightarrow 3 \leq 3$ (vẫn thỏa mãn).
  - Tuy nhiên, nếu ta chọn $h(S) = 4$ và $h(C) = 1$ thì xét cạnh $S \to C$: $4 \leq 2 + 1 \Leftrightarrow 4 \leq 3$ (Sai → không consistent, nhưng vẫn admissible vì $4 \leq 4$)